

Bank Management Application (C++)

Project Overview

This project is a console-based Bank Management Application developed in C++. It simulates core banking operations such as account creation, deposit, withdrawal, and balance inquiry using Object-Oriented Programming (OOP) concepts and file handling for persistent storage.

Features

- Create New Bank Account
- Deposit Money
- Withdraw Money
- Check Account Balance
- Display Customer Details
- Store Customer Records in File
- Persistent Data Storage using Binary File Handling

Technologies Used

Technology	Usage
C++	Programming Language
Object-Oriented Programming	Core Design
File Handling (fstream)	Data Storage
Binary File Storage	Persistent Records

OOP Concepts Used

Concept	Usage
Class & Object	BankAccount class
Encapsulation	Private data members
Abstraction	Banking operations through methods
Constructor	Initialize account details
File Handling	Save and retrieve records

Complete C++ Program

```
#include <iostream>
#include <fstream>
#include <cstring>

using namespace std;

class BankAccount {
private:
    int accountNumber;
```

```

        char name[50];
        float balance;

public:
    void createAccount();
    void showAccount() const;
    void deposit(float);
    void withdraw(float);
    int getAccountNumber() const;
    float getBalance() const;
};

void BankAccount::createAccount() {
    cout << "\nEnter Account Number: ";
    cin >> accountNumber;

    cout << "Enter Account Holder Name: ";
    cin.ignore();
    cin.getline(name, 50);

    cout << "Enter Initial Balance: ";
    cin >> balance;

    cout << "\nAccount Created Successfully!\n";
}

void BankAccount::showAccount() const {
    cout << "\nAccount Number : " << accountNumber;
    cout << "\nAccount Holder : " << name;
    cout << "\nBalance          : " << balance << endl;
}

void BankAccount::deposit(float amount) {
    balance += amount;
}

void BankAccount::withdraw(float amount) {
    if(amount <= balance) {
        balance -= amount;
        cout << "\nWithdrawal Successful.\n";
    } else {
        cout << "\nInsufficient Balance!\n";
    }
}

int BankAccount::getAccountNumber() const {
    return accountNumber;
}

float BankAccount::getBalance() const {
    return balance;
}

void writeAccount() {
    BankAccount acc;
    ofstream outFile;

    outFile.open("bank.dat", ios::binary | ios::app);

    acc.createAccount();

    outFile.write(reinterpret_cast<char *>(&acc), sizeof(BankAccount));

    outFile.close();
}

void displayAccount(int accNo) {
    BankAccount acc;
    ifstream inFile;
    bool found = false;

    inFile.open("bank.dat", ios::binary);

    if(!inFile) {
        cout << "\nFile could not be opened!\n";
        return;
    }

    while(inFile.read(reinterpret_cast<char *>(&acc), sizeof(BankAccount))) {

```

```

        if(acc.getAccountNumber() == accNo) {
            acc.showAccount();
            found = true;
        }
    }

    inFile.close();

    if(!found)
        cout << "\nAccount Not Found!\n";
}

void modifyAccount(int accNo, int option) {
    BankAccount acc;
    fstream file;
    bool found = false;

    file.open("bank.dat", ios::binary | ios::in | ios::out);

    if(!file) {
        cout << "\nFile could not be opened!\n";
        return;
    }

    while(!file.eof() && found == false) {
        streampos pos = file.tellg();

        file.read(reinterpret_cast<char *>(&acc), sizeof(BankAccount));

        if(acc.getAccountNumber() == accNo) {
            acc.showAccount();

            float amount;

            if(option == 1) {
                cout << "\nEnter Amount to Deposit: ";
                cin >> amount;
                acc.deposit(amount);
            }

            if(option == 2) {
                cout << "\nEnter Amount to Withdraw: ";
                cin >> amount;
                acc.withdraw(amount);
            }

            file.seekp(pos);
            file.write(reinterpret_cast<char *>(&acc), sizeof(BankAccount));

            cout << "\nRecord Updated Successfully!\n";
            found = true;
        }
    }

    file.close();

    if(!found)
        cout << "\nAccount Not Found!\n";
}

int main() {
    int choice;
    int accNo;

    do {
        cout << "\n\n===== BANK MANAGEMENT SYSTEM =====\n";
        cout << "\n1. Create Account";
        cout << "\n2. Deposit Amount";
        cout << "\n3. Withdraw Amount";
        cout << "\n4. Balance Inquiry";
        cout << "\n5. Exit";
        cout << "\nEnter Your Choice: ";
        cin >> choice;

        switch(choice) {
            case 1:
                writeAccount();
                break;
            case 2:

```

```

        cout << "\nEnter Account Number: ";
        cin >> accNo;
        modifyAccount(accNo, 1);
        break;

    case 3:
        cout << "\nEnter Account Number: ";
        cin >> accNo;
        modifyAccount(accNo, 2);
        break;

    case 4:
        cout << "\nEnter Account Number: ";
        cin >> accNo;
        displayAccount(accNo);
        break;

    case 5:
        cout << "\nThank You for Using Bank Management System!\n";
        break;

    default:
        cout << "\nInvalid Choice!\n";
    }

    } while(choice != 5);

    return 0;
}

```

Sample Output

```

===== BANK MANAGEMENT SYSTEM =====
1. Create Account
2. Deposit Amount
3. Withdraw Amount
4. Balance Inquiry
5. Exit

Enter Your Choice: 1

Enter Account Number: 101
Enter Account Holder Name: Rahul
Enter Initial Balance: 5000

Account Created Successfully!

```

File Used

File Name	Purpose
bank.dat	Stores customer account records permanently

How to Compile and Run

```

g++ bank.cpp -o bank
./bank

```

Advantages of the System

- Easy to use
- Secure storage using files
- Persistent customer records
- Prevents withdrawal beyond balance

- Demonstrates practical OOP implementation

Future Enhancements

- Password Authentication
- Admin Login
- Interest Calculation
- Transaction History
- GUI Interface
- Database Integration (MySQL)

Conclusion

The Bank Management Application successfully demonstrates the implementation of banking operations using C++ with OOP concepts and file handling. It provides a simple, secure, and persistent system for managing bank accounts and transactions.